1 Contest 1
2 What to use when 1
3 Techniques 2
4 Mathematics 4
5 Data structures 6
6 Numerical 7
7 Number theory 7
8 Combinatorial 8
9 Graph 8
10 Geometry 10
11 Strings 12
12 Various 12

Contest (1)

troubleshoot.txt 52 lines

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
```

template.cpp 14 lines

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int main() {
  cin.tie(0)->sync_with_stdio(0);
  cin.exceptions(cin.failbit);
}
```

Makefile 21 lines

```
TARGET = A
FLAGS = -std=c++20 -Wall -Wconversion -Wfatal-errors -g -
    fsanitize=undefined,address

all: $(TARGET)

$(TARGET): $(TARGET).cpp
  g++ $(FLAGS) $(TARGET).cpp -o $(TARGET)

run: $(TARGET)
  ./$(TARGET)

test: $(TARGET)
  @for f in $(TARGET)*.in; do \
    echo "== $$f expected:"; cat $${f%.in}.ans; \
    echo "-- got:"; ./$(TARGET) < $$f | tee out; echo; \
    diff -b out $${f%.in}.ans > /dev/null && echo "-- ok" ||
        echo "-- WRONG"; \
  done

hash: # make hash < file: hash ignoring whitespace and comments
  @cpp -dD -P -fpreprocessed | tr -d '[:space:]' \
    | md5sum | cut -c-6
```

What to use when (2)

2.1 From the constraints

About 10^8 simple operations run in a second. With many test cases, look at the sum of n.

$n \leq$	Aim for	Often
10	$O(n!)$	try every order (`next_permutation`)
20	$O(2^n n)$	try every subset, bitmask DP
40	$O(2^{n/2})$	meet in the middle: solve each half, combine
500	$O(n^3)$	Floyd–Warshall, interval DP
5000	$O(n^2)$	DP over pairs (LCS, knapsack)
$2 \cdot 10^5$	$O(n \log n)$	sorting, binary search, segment tree
10^7	$O(n)$	prefix sums, two pointers, sieve
10^{18}	$O(\log n)$	a formula, binary search, matrix power

2.2 From the wording

2.2.1 Optimising and searching

- *minimise the maximum, maximise the minimum, smallest x that works* → binary search on the answer, if every x above a working one also works (`BinarySearchAnswer`).
- *maximise or minimise something that rises then falls, e.g. a distance over time* → ternary search (`TernarySearch`).
- *choose the best order* → for $n \leq 10$ try all; otherwise sort by some key, and check the greedy against brute force on small cases.
- *most intervals that do not overlap* → sort by end, take each one that starts after the last taken ends.
- *the x minimising $\sum |x - a_i|$* → the median; for $\sum (x - a_i)^2$, the mean.

2.2.2 Counting and DP

- *how many ways..., modulo $10^9 + 7$* → DP, or combinatorics (`ModBinomial`).
- *pick items within a weight limit* → knapsack; *make an amount from coins*: `coinWays`, `minCoins` (`Knapsack`).
- *fewest edits between strings, common subsequence* → `LCS`; *increasing subsequence*: `LIS`.
- *visit all of $n \leq 18$ places, assign n people to n tasks* → bitmask DP (`BitmaskDP`).
- *merge or split a sequence, cost per segment* → interval DP (`IntervalDP`).
- *count or longest paths in a DAG* → DP in topological order (`TopoSort`).
- *two players, both play optimally* → DP over positions (see Techniques).
- *expected value* → linearity: add the expectations of the parts; long run: `SteadyState`.

2.2.3 Graphs

- *fewest moves (grid, puzzle)* → BFS where nodes are states (`GraphBasics`).
- *shortest path with extra state (fuel, keys held, last move)* → Dijkstra or BFS on (node, state) pairs.
- *cheapest route* → weights ≥ 0: `Dijkstra`; negative: `BellmanFord`; all pairs, $n \leq 500$: `FloydWarshall`.
- *exchange rates: can you make money?* → weights $-\log(\text{rate})$, look for a negative cycle (`BellmanFord`, with `double` weights).
- *tasks with prerequisites, is there a deadlock?* → `TopoSort` (a cycle iff fewer than n nodes come out).
- *connect everything as cheaply as possible* → `Kruskal`. The path between two nodes in the MST also has the smallest possible largest edge.
- *are a and b connected, as links are added* → `UnionFind`. Links removed over time: go backwards, adding them.
- *split into two sides with no conflicts* → `Bipartite`.
- *pair up one-to-one (people and jobs)* → matching (`Dinic`). Fewest nodes touching every edge of a bipartite graph = maximum matching.
- *fewest edges to cut to separate s from t, max throughput* → min cut = max flow (`Dinic`).

2.2.4 Trees

- *is it a tree?* → connected and $m = n - 1$.
- *distance between two nodes, common ancestor, jumping up k* → `LCA`.
- *longest path (diameter)* → BFS from any node to the farthest node u, then BFS from u.

2.2.5 Arrays and ranges

- *sum of a[l..r], no updates* → `PrefixSums` (also 2D, and xor). *Add to ranges, read at the end*: difference array.
- *update one value, range sums* → `FenwickTree`; range min or max: `SegmentTree`.
- *add to ranges and query ranges* → `LazySegmentTree`.
- *longest or shortest subarray with a sum limit (values ≥ 0), at most k distinct* → `TwoPointers`.
- *number of subarrays with sum k* → `countSum` (`PrefixSums`).
- *largest subarray sum* → Kadane: `cur = max(a[i], cur + a[i])`, answer is the largest `cur`.
- *count pairs $i < j$ with $a_i > a_j$ (inversions)* → `FenwickTree` over compressed values.
- *values up to 10^9, only their order matters* → coordinate compression.
- *nearest bigger or smaller element* → monotonic stack.

2.2.6 Strings

- *find all occurrences, smallest period* → `KMP`.
- *compare substrings, longest substring that repeats or is common to two strings* → `Hashing`.
- *palindromes* → `Manacher`, or compare forward and backward hashes.

2.2.7 Maths

- $a^b \bmod p$, *dividing mod p,* $\binom{n}{r} \bmod p$ → `ModBinomial`.
- *primes up to 10^7* → `Eratosthenes`; one $n \leq 10^{12}$: trial division up to $\sqrt{n}$.
- *a linear recurrence for n up to 10^{18}* → matrix power (`Matrix`).
- *solve linear equations* → `SolveLinear`.

2.2.8 Geometry

- *area, point inside* → `PolygonArea`, `InsidePolygon`.
- *shortest fence around points, two farthest points* → `ConvexHull`, `HullDiameter`.
- *closest two points, smallest circle around points* → `ClosestPair`, `MinimumEnclosingCircle`. Use `ll` where you can; compare doubles with a tolerance.

2.3 If nothing fits

Solve small cases by hand or with a brute force, and look for a pattern. Fix one thing (the maximum, the first element, the meeting point) and see if the rest becomes easy. Go backwards in time. Sort the input. Ask whether the answer is monotone in something (binary search). Turn states and moves into a graph.

Techniques (3)

3.1 Searching

BinarySearchAnswer.cpp

Description: Binary search on the answer, for "the smallest x that works" when works(x) goes false, ..., false, true, ..., true (if x works, so does every bigger x). firstTrue returns the smallest x in [lo, hi) that works, or hi if none does; each step halves the range. Take mid = lo + (hi - lo) / 2, not (lo + hi) / 2: the sum can overflow for big values, and for negative ones it rounds towards 0, so mid can equal hi and the loop never ends (e.g. lo = -3, hi = -2). For "the largest x that works" (true, ..., true, false, ...), find the first x that does not work and subtract 1. firstTrueReal does the same on real numbers with a fixed 100 halvings, so there is no off-by-one. For a sorted array, use lower_bound instead (BinarySearch.cpp). minMaxPart is an example: cut a (all a[i] ≥ 0) into at most k contiguous parts, minimising the largest part's sum. Checking one cap greedily is easy, while finding the best cap directly is not.

Usage: `ll x = firstTrue(lo, hi, works); // works: a lambda taking ll, returning bool`

Time: $\mathcal{O}(\log(hi - lo))$ calls to works

589aa1, 30 lines

```
template<class F> ll firstTrue(ll lo, ll hi, F works) {
  while (lo < hi) {
    ll mid = lo + (hi - lo) / 2;
    if (works(mid)) hi = mid;
    else lo = mid + 1;
  }
  return lo;
}
template<class F> double firstTrueReal(double lo, double hi, F
    works) {
  rep(it,0,100) {
    double mid = (lo + hi) / 2;
    if (works(mid)) hi = mid;
    else lo = mid;
  }
  return hi;
}

ll minMaxPart(const vector<ll>& a, int k) {
  auto works = [&](ll cap) { // can every part have sum <= cap?
    int parts = 1;
    ll cur = 0;
    for (ll x : a) {
      if (x > cap) return false;
      if (cur + x > cap) parts++, cur = 0;
      cur += x;
    }
    return parts <= k;
  };
  return firstTrue(0, accumulate(all(a), 0LL), works);
}
```

TernarySearch.cpp

Description: Ternary search: finds the x in [lo, hi] where f is largest, when f strictly increases up to its peak and never increases after it (unimodal). Each step compares f at two points and drops the third of the range that cannot contain the peak. For the smallest value instead, use -f. Typical uses: a distance that shrinks and then grows over time, or a sum of convex functions (minimum). Flat stretches are fine after the peak but break it before the peak. ternaryMax evaluates f 200 times; each step keeps 2/3 of the range, so 100 steps shrink it by 10^{-17}. ternaryMaxInt is for integers: the peak is the first x with $f(x) \geq f(x+1)$, found by binary search.

Usage: `double x = ternaryMax(lo, hi, f); // f: a lambda taking and returning double`

Time: $\mathcal{O}(\log(hi - lo))$ for integers

c4eee5, 16 lines

```
template<class F> double ternaryMax(double lo, double hi, F f)
    {
  rep(it,0,100) {
    double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
    if (f(m1) < f(m2)) lo = m1; // the peak is right of m1
    else hi = m2;               // the peak is left of m2
  }
  return (lo + hi) / 2;
}
template<class F> ll ternaryMaxInt(ll lo, ll hi, F f) {
  while (lo < hi) {
    ll mid = lo + (hi - lo) / 2;
    if (f(mid) < f(mid + 1)) lo = mid + 1;
    else hi = mid;
  }
  return lo;
}
```

3.2 Arrays

PrefixSums.cpp

Description: Prefix sums: p[i] = a[0] + ... + a[i-1], so the sum of a[l..r) is p[r] - p[l] in O(1). In 2D, P[i][j] is the sum of g over rows $< i$ and columns $< j$, and any rectangle follows by inclusion-exclusion. countSum counts the subarrays with sum exactly k in $O(n \log n)$: pairs of prefix sums with p[r] - p[l] = k (works with negatives). Difference array: to add x to a[l..r) many times and read a only at the end, do d[l] += x and d[r] -= x (d has size n + 1), then add the running sum d[0] + ... + d[i] to each a[i]. For both online, use a Fenwick tree.

Usage: `vector<ll> p = prefixSums(a); ll s = p[r] - p[l];`
`auto P = prefixSums2D(g); // rows [r1, r2), columns [c1, c2):`
`ll rect = P[r2][c2] - P[r1][c2] - P[r2][c1] + P[r1][c1];`

Time: $\mathcal{O}(n)$ or $\mathcal{O}(nm)$ setup, $\mathcal{O}(1)$ per query

ec9aa9, 22 lines

```
vector<ll> prefixSums(const vector<ll>& a) {
  vector<ll> p(sz(a) + 1);
  rep(i,0,sz(a)) p[i+1] = p[i] + a[i];
  return p;
}
vector<vector<ll>> prefixSums2D(const vector<vector<ll>>& g) {
  int n = sz(g), m = n ? sz(g[0]) : 0;
  vector<vector<ll>> P(n + 1, vector<ll>(m + 1));
  rep(i,0,n) rep(j,0,m)
    P[i+1][j+1] = g[i][j] + P[i][j+1] + P[i+1][j] - P[i][j];
  return P;
}
ll countSum(const vector<ll>& a, ll k) {
  map<ll, ll> seen{{0, 1}}; // prefix sum -> times seen
  ll s = 0, cnt = 0;
  for (ll x : a) {
    s += x;
    if (seen.count(s - k)) cnt += seen[s - k];
    seen[s]++;
  }
  return cnt;
}
```

TwoPointers.cpp

Description: Two pointers (sliding window): for each right end r, move the left end l forward until the window a[l..r] is right again. This works when shrinking a good window keeps it good (and growing a bad one keeps it bad), e.g. sum $\le$ S with all a[i] $\ge$ 0, or at most k distinct values: then l only moves forward, so the whole loop is O(n). Both functions need all a[i] $\ge$ 0. longestWindow is the length of the longest subarray with sum $\le$ S (0 if none). shortestWindow is the length of the shortest subarray with sum $\ge$ S $>$ 0 (-1 if none).
Time: $\mathcal{O}(n)$

61177e, 20 lines

```
int longestWindow(const vector<ll>& a, ll S) {
  int best = 0;
  ll sum = 0;
  for (int l = 0, r = 0; r < sz(a); r++) {
    sum += a[r];
    while (l <= r && sum > S) sum -= a[l++];
    best = max(best, r - l + 1);
  }
  return best;
}
int shortestWindow(const vector<ll>& a, ll S) {
  int best = INT_MAX;
  ll sum = 0;
  for (int l = 0, r = 0; r < sz(a); r++) {
    sum += a[r];
    while (l < r && sum - a[l] >= S) sum -= a[l++]; // a[l] not
        needed
    if (sum >= S) best = min(best, r - l + 1);
  }
  return best == INT_MAX ? -1 : best;
}
```

Coordinate compression. If values are up to 10^9 but only their order matters, replace each by its index among the sorted distinct values: `vals = a; sort, erase(unique); x = lower_bound(all(vals), x) - vals.begin()`.

Monotonic stack. For the nearest bigger element to the left of each a[i], keep a stack of indices: pop while `a[top] <= a[i]`, then the top (if any) is the answer, then push i. $O(n)$ in total.

3.3 Dynamic programming

Designing a DP. (1) State: the few facts about a partial solution that decide what can still happen, e.g. `dp[i][c]` = the best using the first i items with capacity c. (2) Transition: get a state from smaller ones, usually by trying each choice for the next step. (3) Base cases. (4) An order in which every state comes after the states it uses, or recursion with memoisation, which finds the order by itself. (5) Where the answer is, e.g. `dp[n][W]`. The time is the number of states times the work per state: check it against the constraints. Counting problems add over the choices (mod p), optimisation problems take the min or max.

Common states: a prefix, `dp[i]` (e.g. grid paths: `dp[i][j] = dp[i-1][j] + dp[i][j-1]`; longest increasing subsequence: `LIS.h`); a node of a DAG, in topological order; a subset of up to 20 items, `dp[S]`; a segment, `dp[l][r]`. Games where both players play optimally: a position is winning iff some move leads to a losing position (usually, having no move loses).

Knapsack.cpp

Description: Knapsack DPs over the capacity c = 0..W. items holds (weight, value) pairs. knapsack01: each item at most once; best[c] is the most value with total weight $\le$ c. c goes down, so best[c - w] does not include the current item yet. knapsackUnbounded: any number of each item; c goes up. coinWays[c] is the number of ways to make exactly c (order does not matter: coins in the outer loop; reduce mod p if asked). minCoins[c] is the fewest coins making exactly c, LLONG_MAX if impossible. If W is too big but values are small, swap the roles: the least weight for each total value.
Usage: `vector<ll> best = knapsack01(W, items); ll ans = best[W];`
Time: $\mathcal{O}(nW)$

91b669, 28 lines

```
vector<ll> knapsack01(int W, const vector<pii>& items) {
  vector<ll> best(W + 1);
  for (auto [w, v] : items)
    for (int c = W; c >= w; c--)
      best[c] = max(best[c], best[c - w] + v);
  return best;
}
vector<ll> knapsackUnbounded(int W, const vector<pii>& items) {
  vector<ll> best(W + 1);
  for (auto [w, v] : items)
    for (int c = w; c <= W; c++)
      best[c] = max(best[c], best[c - w] + v);
  return best;
}
vector<ll> coinWays(int W, const vi& coins) {
  vector<ll> ways(W + 1);
  ways[0] = 1;
  for (int x : coins)
    for (int c = x; c <= W; c++) ways[c] += ways[c - x];
  return ways;
}
vector<ll> minCoins(int W, const vi& coins) {
  vector<ll> f(W + 1, LLONG_MAX);
  f[0] = 0;
  rep(c,1,W+1) for (int x : coins)
    if (x <= c && f[c - x] != LLONG_MAX) f[c] = min(f[c], f[c -
        x] + 1);
  return f;
}
```

LCS.cpp

Description: DP over prefixes of two strings: dp[i][j] is the answer for a[0..i) and b[0..j). lcs is the length of the longest common subsequence (in order, not necessarily contiguous): if the last characters match, use them, otherwise drop one of them. editDistance is the fewest single-character inserts, deletes and replacements turning a into b. To print a solution, walk back from dp[n][m], each time taking a move that gives the stored value.
Time: $\mathcal{O}(nm)$

988487, 18 lines

```
int lcs(const string& a, const string& b) {
  int n = sz(a), m = sz(b);
  vector<vi> dp(n + 1, vi(m + 1));
  rep(i,1,n+1) rep(j,1,m+1)
    dp[i][j] = a[i-1] == b[j-1] ? dp[i-1][j-1] + 1
                                : max(dp[i-1][j], dp[i][j-1]);
  return dp[n][m];
}
int editDistance(const string& a, const string& b) {
  int n = sz(a), m = sz(b);
  vector<vi> dp(n + 1, vi(m + 1));
  rep(i,0,n+1) dp[i][0] = i; // delete all of a[0..i)
  rep(j,0,m+1) dp[0][j] = j; // insert all of b[0..j)
  rep(i,1,n+1) rep(j,1,m+1)
    dp[i][j] = min({dp[i-1][j] + 1, dp[i][j-1] + 1,
                    dp[i-1][j-1] + (a[i-1] != b[j-1])});
  return dp[n][m];
}
```

BitmaskDP.cpp

Description: Bitmask DP over subsets, for $1 \le n \le$ about 18: a set S of items is the int whose bit i is set iff i is in S. tsp is the travelling salesman problem: dp[S][v] is the cheapest path that starts at 0, visits exactly the nodes in S and ends at v; tsp returns the cheapest round trip through all nodes. The same pattern assigns n people to n tasks: dp[S] is the best way to give the first popcount(S) people the tasks in S. Memory is $2^n n$ values, so use int if the costs fit.
Usage: `ll cost = tsp(d); // d[u][v] = cost from u to v, big but finite (not LLONG_MAX) if no edge`
Time: $\mathcal{O}(2^n n^2)$

79e14f, 14 lines

```
ll tsp(const vector<vector<ll>>& d) {
  int n = sz(d), full = (1 << n) - 1;
  vector<vector<ll>> dp(1 << n, vector<ll>(n, LLONG_MAX));
  dp[1][0] = 0;
  rep(S,1,full+1) rep(v,0,n) if (dp[S][v] != LLONG_MAX)
    rep(u,0,n) if (!(S >> u & 1)) {
      ll& to = dp[S | 1 << u][u];
      to = min(to, dp[S][v] + d[v][u]);
    }
  ll best = LLONG_MAX;
  rep(v,0,n) if (dp[full][v] != LLONG_MAX)
    best = min(best, dp[full][v] + d[v][0]);
  return best;
}
```

IntervalDP.cpp

Description: Interval DP: dp[l][r] is the answer for the segment a[l..r), found by trying every split point m and combining dp[l][m] and dp[m][r]. Go by increasing length, so both parts are already done. minMergeCost: n piles in a row are merged into one, two neighbouring piles at a time, and each merge costs the total size of the two piles; returns the cheapest total. The same shape solves matrix chain multiplication, optimal bracketing and many "remove or merge parts of a sequence" problems.
Time: $\mathcal{O}(n^3)$, so n up to about 500

8eb404, 13 lines

```
ll minMergeCost(const vector<ll>& a) {
  int n = sz(a);
  vector<ll> p(n + 1);
  rep(i,0,n) p[i+1] = p[i] + a[i];
  vector<vector<ll>> dp(n + 1, vector<ll>(n + 1));
```

```
  rep(len,2,n+1) for (int l = 0; l + len <= n; l++) {
    int r = l + len;
    dp[l][r] = LLONG_MAX;
    rep(m,l+1,r) dp[l][r] = min(dp[l][r], dp[l][m] + dp[m][r]);
    dp[l][r] += p[r] - p[l]; // the last merge
  }
  return n ? dp[0][n] : 0;
}
```

Mathematics (4)

4.1 Standard library

StdMath.cpp

Description: C++20, all in bits/stdc++.h. Here a, b are int/ll, u is unsigned/ull (the <bit> functions only take unsigned) and x, y, t are double. long is 32-bit on Windows (Codeforces): use long long. 0fcf02, 30 lines

```
gcd(a, b); lcm(a, b);
__lg(a);            // floor(log2(a)), exact
bit_width(u);       // floor(log2(u)) + 1
bit_ceil(u);        // smallest power of 2 >= u
has_single_bit(u); popcount(u); __builtin_popcountll(a);
countl_zero(u); countr_zero(u); rotl(u, k); rotr(u, k);
log2(x);            // inexact for big integers
atan2(y, x);        // angle of (x, y) in [-pi, pi]
hypot(x, y); sqrt(x); cbrt(x);
midpoint(a, b);     // (a + b) / 2 without overflow
lerp(x, y, t);      // x + t * (y - x)

INT_MAX    == 2147483647;             // ~2.1e9
UINT_MAX   == 4294967295;             // ~4.3e9
LLONG_MAX  == 9223372036854775807;    // ~9.2e18
ULLONG_MAX == 18446744073709551615;   // ~1.8e19
INT_MIN == -INT_MAX - 1; LLONG_MIN == -LLONG_MAX - 1;
numeric_limits<ll>::max(); numeric_limits<ll>::min();
__int128 big;                         // ~1.7e38, no cin/cout
DBL_DIG == 15; LDBL_DIG == 18;        // significant digits
DBL_MAX;                              // ~1.8e308

accumulate(all(v), 0LL);                  // 0 would sum in int
partial_sum(all(v), p.begin());           // v[0] + ... + v[i]
adjacent_difference(all(v), d.begin()); // v[i] - v[i-1]
sort(all(v)); do {} while (next_permutation(all(v)));
prev_permutation(all(v));                 // false on wrap-around

numbers::pi; numbers::e; numbers::phi; numbers::sqrt2;
numbers::pi_v<long double>;
```

4.2 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \Rightarrow \begin{aligned} x &= \frac{ed - bf}{ad - bc} \\ y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A_i'}{\det A}$$

where A_i' is A with the i'th column replaced by b.

4.3 Recurrences

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \ldots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \ldots, d_k$ s.t.

$$a_n = d_1 r_1^n + \cdots + d_k r_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

4.4 Trigonometry

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$
$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$
$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$
$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$
$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$a \cos x + b \sin x = r \cos(x - \phi)$$
$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}, \phi = \operatorname{atan2}(b, a)$.

4.5 Geometry

4.5.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b + c}\right)^2\right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

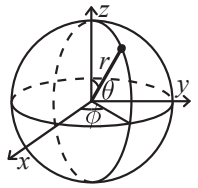
4.5.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

4.5.3 Spherical coordinates



$$\begin{aligned} x &= r \sin\theta \cos\phi \\ y &= r \sin\theta \sin\phi \\ z &= r \cos\theta \end{aligned} \qquad \begin{aligned} r &= \sqrt{x^2 + y^2 + z^2} \\ \theta &= \operatorname{acos}(z/\sqrt{x^2 + y^2 + z^2}) \\ \phi &= \operatorname{atan2}(y, x) \end{aligned}$$

4.6 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$
$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$
$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$
$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

4.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$

$$\sqrt{1 + x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

4.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y,

$$V(aX + bY) = a^2 V(X) + b^2 V(Y).$$

4.8.1 Counting

Ways to pick r of n items: $\binom{n}{r} = \frac{n!}{r!(n-r)!}$ unordered, ${}^nP_r = \frac{n!}{(n-r)!}$ ordered, and $\binom{n+r-1}{r}$ unordered with repetition. Also $\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}$.

Answers are often mod $p = 10^9 + 7$ or 998 244 353 (both prime). Reduce after every $+$, $-$ and $\cdot$ using `ll`, and write `(a - b + p) % p`, as `%` can be negative. To divide by b, multiply by $b^{-1} \equiv b^{p-2}$ (Fermat).

ModBinomial.cpp
Description: $\binom{n}{r}$ and ${}^nP_r$ mod a prime, 0 if $r < 0$ or $r > n$.
Time: $\mathcal{O}(N)$ for init, $\mathcal{O}(1)$ per query
92ec98, 23 lines

```
const ll mod = 1e9 + 7; // or 998244353
ll modpow(ll b, ll e) { // b^e, and 1/b if e = mod - 2
  ll r = 1;
  for (b %= mod; e; e /= 2, b = b * b % mod)
    if (e & 1) r = r * b % mod;
  return r;
}
const int N = 1e6;
ll f[N + 1], fi[N + 1]; // n! and 1/n!
void init() { // call once
  f[0] = 1;
  rep(i,1,N+1) f[i] = f[i-1] * i % mod;
  fi[N] = modpow(f[N], mod - 2);
  for (int i = N; i > 0; i--) fi[i-1] = fi[i] * i % mod;
}
ll nCr(int n, int r) {
  if (r < 0 || r > n) return 0;
  return f[n] * fi[r] % mod * fi[n-r] % mod;
}
ll nPr(int n, int r) {
  if (r < 0 || r > n) return 0;
  return f[n] * fi[n-r] % mod;
}
```

4.8.2 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\mathrm{Bin}(n, p)$, $n = 1, 2, \ldots$, $0 \le p \le 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

$\mathrm{Bin}(n, p)$ is approximately $\mathrm{Po}(np)$ for small p.

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\mathrm{Fs}(p)$, $0 \le p \le 1$.

$$p(k) = p(1-p)^{k-1},\ k = 1, 2, \ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\mathrm{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda}\frac{\lambda^k}{k!}, k = 0, 1, 2, \ldots$$

$$\mu = \lambda,\ \sigma^2 = \lambda$$

4.8.3 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\mathrm{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2},\ \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\mathrm{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \ge 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

4.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. With transition matrix $\mathbf{P} = (p_{ij})$, $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$ (columns sum to 1), the distribution after n steps is $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$, where $\mathbf{p}^{(0)}$ is the initial distribution. Compute it in $O(S^3 \log n)$ for S states with `Matrix.h`: `(P^n) * p0` (the brackets are needed, as $*$ binds tighter than `^`). With doubles the round-off grows like $n \cdot 10^{-16}$, so for big n rescale each column to sum 1 after every multiply, as `longRun` below does.

Steady state. π is stationary if $\mathbf{P}\pi = \pi$ and $\sum_i \pi_i = 1$. It is unique iff the chain has one closed class (set of states it cannot leave), e.g. if it is *irreducible* (every state can reach every state); then $\pi_i = 1/\mathbb{E}(T_i)$, where T_i is the time between two visits to state i. If it is also *aperiodic* (the gcd of cycle lengths is 1), $\mathbf{p}^{(n)} \to \pi$ from any start; if it is periodic, $\mathbf{p}^{(n)}$ keeps cycling and π is its average over time. With several closed classes, where the chain ends up depends on $\mathbf{p}^{(0)}$. `stationary` below finds a unique π exactly; `longRun` handles every case.

Diagonalising. If $\mathbf{P} = VDV^{-1}$ with $D = \mathrm{diag}(\lambda_1, \ldots, \lambda_S)$ and eigenvectors $\mathbf{v}_i$ as the columns of V, then $\mathbf{P}^n = VD^nV^{-1}$, so writing $\mathbf{p}^{(0)} = \sum_i c_i \mathbf{v}_i$ gives $\mathbf{p}^{(n)} = \sum_i c_i \lambda_i^n \mathbf{v}_i$. $\lambda_1 = 1$ gives π; in an irreducible aperiodic chain all other $|\lambda_i| < 1$, so their terms decay like $|\lambda_2|^n$. For two states with $a = \Pr(0 \to 1)$, $b = \Pr(1 \to 0)$:

$$\pi = \tfrac{1}{a+b}(b, a), \quad \mathbf{p}^{(n)} = \pi + (1 - a - b)^n(\mathbf{p}^{(0)} - \pi).$$

SteadyState.cpp
Description: Steady state of a Markov chain, where P.d[i][j] = $\Pr(j \to i)$. stationary finds π exactly if it is unique (one closed class, e.g. irreducible; periodic is fine), and returns false otherwise. It solves $(P - I)\pi = 0$. longRun works for any chain and returns the long-run distribution from p0: the average over time if periodic, and depending on p0 if there are several closed classes. It computes $((P + I)/2)^{2^{60}}$, which is aperiodic with the same stationary distributions, and rescales each column to sum 1 against round-off.
Time: $\mathcal{O}(S^3)$
"../data-structures/Matrix.h", "../numerical/SolveLinear.h" 11374e, 23 lines

```
const int S = 3; // number of states
typedef array<double, S> V;
V longRun(Matrix<double, S> P, V p0) {
```

```
  rep(i,0,S) rep(j,0,S) P.d[i][j] = (P.d[i][j] + (i == j)) / 2;
  rep(k,0,60) {
    P = P * P;
    rep(j,0,S) {
      double t = 0;
      rep(i,0,S) t += P.d[i][j];
      rep(i,0,S) P.d[i][j] /= t;
    }
  }
  return P * p0;
}
bool stationary(Matrix<double, S> P, V& pi) {
  vector<vd> A(S, vd(S)); vd b(S), x(S);
  rep(i,0,S) rep(j,0,S) A[i][j] = P.d[i][j] - (i == j);
  rep(j,0,S) A[S-1][j] = 1; // last row: sum(pi) = 1
  b[S-1] = 1;
  if (solveLinear(A, b, x) < S) return false;
  rep(i,0,S) pi[i] = x[i];
  return true;
}
```

Data structures (5)

StdContainers.cpp

Description: set, multiset and map are sorted, with everything $O(\log n)$. Use their own s.lower_bound(x), as lower_bound(all(s), x) is $O(n)$ on them. unordered_set/map are $O(1)$ on average, but slow on anti-hash tests. pair and tuple compare lexicographically.

dcb762, 34 lines

```
vector<T> v(n, val);
v.push_back(x); v.pop_back(); v.back(); v.resize(n);
v.insert(v.begin() + i, x); v.erase(v.begin() + i); // O(n)
sort(all(v)); v.erase(unique(all(v)), v.end());     // dedupe

set<T> s; multiset<T> ms;
s.insert(x); s.erase(x); s.count(x); s.contains(x);
*s.begin(); *s.rbegin();                // min, max
s.lower_bound(x); s.upper_bound(x);
ms.erase(x);                            // erases ALL copies of x
ms.erase(ms.find(x));                   // one copy, x must exist

map<K, V> m;
m[k]++;                                 // inserts V() if missing
m.count(k); m.erase(k); m.lower_bound(k);
for (auto& [key, val] : m) {}           // in key order
unordered_set<T> us; unordered_map<K, V> um;

priority_queue<T> pq;                   // max-heap
pq.push(x); pq.top(); pq.pop(); pq.size();
priority_queue<T, vector<T>, greater<T>> minpq;
deque<T> dq;                            // dq[i] O(1)
dq.push_front(x); dq.pop_front(); dq.push_back(x); dq.front();
queue<T> q; q.push(x); q.front(); q.pop();
stack<T> st; st.push(x); st.top(); st.pop();

bitset<N> b;                            // operations O(N / 64)
b[i] = 1; b.count(); b.set(); b.reset(); b.flip(); b <<= k;
b._Find_first(); b._Find_next(i);   // next set bit, N if none

string str;
str.substr(pos, len); str.find(t) == string::npos;
to_string(x); stoi(str); stoll(str);
auto [fst, snd] = pr;
```

BinarySearch.cpp

Description: Binary search on a sorted vector v. lower_bound and upper_bound return v.end() if there is no such element (then the index is v.size()), so check before using *it. x occurs $j - i$ times, the last element $< x$ is at $i - 1$ and the last $\leq x$ at $j - 1$. partition_point returns the first element for which the condition is false; the condition must be true on a prefix of v and false after it. For a vector sorted in descending order, pass greater().

Time: $\mathcal{O}(\log n)$

5ede26, 10 lines

```
sort(all(v));
auto it = lower_bound(all(v), x);  // first >= x
auto it2 = upper_bound(all(v), x); // first > x
if (it != v.end() && *it == x) {}  // x is at it - v.begin()
if (it2 != v.end()) {}             // smallest > x is *it2
auto i = it - v.begin(), j = it2 - v.begin();
bool found = binary_search(all(v), x);
auto k = partition_point(all(vp),  // first p.first >= x
  [&](auto& p) { return p.first < x; }) - vp.begin();
auto i2 = lower_bound(all(desc), x, greater()) - desc.begin();
```

FenwickTree.cpp

Description: Fenwick tree (binary indexed tree): change one element and get prefix sums, both in $O(\log n)$. Shorter and faster than a segment tree, but only for sums (or anything you can undo, like xor). add(i, x) does a[i] += x, and sum(r) is $a_0 + \cdots + a_{r-1}$ (0-indexed), so the sum of a[l..r) is sum(r) - sum(l). For range add and point query instead: add x at l and -x at r, then a[i] is sum(i + 1).

Usage: `Fenwick f(n); f.add(i, x); ll s = f.sum(r) - f.sum(l);`

Time: $\mathcal{O}(\log n)$

c704ee, 12 lines

```
struct Fenwick {
  vector<ll> t; // t[i] = a[i - (i & -i)] + ... + a[i - 1]
  Fenwick(int n) : t(n + 1) {}
  void add(int i, ll x) {
    for (i++; i < sz(t); i += i & -i) t[i] += x;
  }
  ll sum(int r) {
    ll s = 0;
    for (; r > 0; r -= r & -r) s += t[r];
    return s;
  }
};
```

SegmentTree.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.

Time: $\mathcal{O}(\log N)$

0f4bdb, 19 lines

```
struct Tree {
  typedef int T;
  static constexpr T unit = INT_MIN;
  T f(T a, T b) { return max(a, b); } // (any associative fn)
  vector<T> s; int n;
  Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
  void update(int pos, T val) {
    for (s[pos += n] = val; pos /= 2;)
      s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
  }
  T query(int b, int e) { // query [b, e)
    T ra = unit, rb = unit;
    for (b += n, e += n; b < e; b /= 2, e /= 2) {
      if (b % 2) ra = f(ra, s[b++]);
      if (e % 2) rb = f(s[--e], rb);
    }
    return f(ra, rb);
  }
};
```

LazySegmentTree.cpp

Description: Segment tree with lazy propagation, for range updates. add(l, r, x) adds x to every a[i] with $l \leq i < r$, and sum(l, r) returns the sum of those a[i]. All a[i] start at 0; to start from an array, call add(i, i + 1, a[i]) for each i. Node v covers [lo, hi) and its children are 2v and 2v + 1. lz[v] is an add already applied to v but not yet to its children: push hands it down before visiting them. For range min instead of sum: apply does t[v] += x (not times the length), combine with min instead of +, and return LLONG_MAX instead of 0 outside the range.

Usage: `LazySeg st(n); st.add(l, r, x); ll s = st.sum(l, r);`

Time: $\mathcal{O}(\log n)$ per operation

061b23, 30 lines

```
struct LazySeg {
  int n;
  vector<ll> t, lz;
  LazySeg(int n) : n(n), t(4 * n), lz(4 * n) {}
  void apply(int v, int lo, int hi, ll x) {
    t[v] += x * (hi - lo), lz[v] += x;
  }
  void push(int v, int lo, int hi) {
    int mid = (lo + hi) / 2;
    apply(2*v, lo, mid, lz[v]), apply(2*v+1, mid, hi, lz[v]);
    lz[v] = 0;
  }
  void add(int v, int lo, int hi, int l, int r, ll x) {
    if (r <= lo || hi <= l) return;
    if (l <= lo && hi <= r) return apply(v, lo, hi, x);
    push(v, lo, hi);
    int mid = (lo + hi) / 2;
    add(2*v, lo, mid, l, r, x), add(2*v+1, mid, hi, l, r, x);
    t[v] = t[2*v] + t[2*v+1];
  }
  ll sum(int v, int lo, int hi, int l, int r) {
    if (r <= lo || hi <= l) return 0;
    if (l <= lo && hi <= r) return t[v];
    push(v, lo, hi);
    int mid = (lo + hi) / 2;
    return sum(2*v, lo, mid, l, r) + sum(2*v+1, mid, hi, l, r);
  }
  void add(int l, int r, ll x) { add(1, 0, n, l, r, x); }
  ll sum(int l, int r) { return sum(1, 0, n, l, r); }
};
```

UnionFind.h

Description: Disjoint-set data structure (union-find). Keeps track of groups (e.g. connected components) as they are merged. UF uf(n) starts with n elements, numbered 0 to n - 1, each in a group of its own (for input numbered 1 to n, subtract 1 from each, or use UF uf(n + 1)). join(a, b) merges the groups of a and b, and returns false if they were already the same group, so the number of groups is n minus the number of joins that returned true. sameSet(a, b) says whether a and b are in the same group, size(x) is the size of x's group, and find(x) is its representative. Internally, e[x] is the parent of x, or minus the group's size if x is the representative.

Usage:
```
UF uf(5); // 0 1 2 3 4, each alone
uf.join(0, 1); // groups {0,1} {2} {3} {4}
uf.join(3, 4); // groups {0,1} {2} {3,4}
uf.sameSet(1, 0); // true
uf.sameSet(1, 3); // false
uf.size(4); // 2
uf.join(1, 0); // false, already together
```

Time: $\mathcal{O}(\alpha(N))$, which is basically constant

7aa27c, 14 lines

```
struct UF {
  vi e;
  UF(int n) : e(n, -1) {}
  bool sameSet(int a, int b) { return find(a) == find(b); }
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
  bool join(int a, int b) {
    a = find(a), b = find(b);
```

```
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$

de4ad0, 21 lines

```
struct RollbackUF {
  vi e; vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

Matrix.h

Description: Basic operations on square matrices.
Usage: `Matrix<int, 3> A;`
`A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};`
`array<int, 3> vec = {1,2,3};`
`vec = (A^N) * vec;`

4da5a2, 26 lines

```
template<class T, int N> struct Matrix {
  typedef Matrix M;
  array<array<T, N>, N> d{};
  M operator*(const M& m) const {
    M a;
    rep(i,0,N) rep(j,0,N)
      rep(k,0,N) a.d[i][k] += d[i][j] * m.d[j][k];
    return a;
  }
  array<T, N> operator*(const array<T, N>& vec) const {
    array<T, N> ret{};
    rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
    return ret;
  }
  M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    while (p) {
      if (p&1) a = a*b;
      b = b*b;
      p >>= 1;
    }
    return a;
  }
};
```

Numerical (6)

6.1 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}(N^3)$

bd5cec, 15 lines

```
double det(vector<vector<double>>& a) {
  int n = sz(a); double res = 1;
  rep(i,0,n) {
    int b = i;
    rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      double v = a[j][i] / a[i][i];
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
    }
  }
  return res;
}
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2 m)$

44c9ab, 38 lines

```
typedef vector<double> vd;
const double eps = 1e-12;

int solveLinear(vector<vd>& A, vd& b, vd& x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n) assert(sz(A[0]) == m);
  vi col(m); iota(all(col), 0);

  rep(i,0,n) {
    double v, bv = 0;
    rep(r,i,n) rep(c,i,m)
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      rep(j,i,n) if (fabs(b[j]) > eps) return -1;
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    rep(j,i+1,n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];
    }
    rank++;
  }

  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n).
Time: $\mathcal{O}(n^3)$

ebfff6, 35 lines

```
int matInv(vector<vector<double>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    double v = A[i][i];
    rep(j,i+1,n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k];
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
    }
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  }

  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  }

  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

Number theory (7)

7.1 Primality

Eratosthenes.cpp

Description: Sieve of Eratosthenes: finds all primes up to MAXP. Call sieve() once; then isPrime[x] says whether $x \leq$ MAXP is prime, and primes lists the primes in increasing order. Each prime crosses out its multiples, starting at i^2 as smaller multiples have a smaller prime factor. There are 664 579 primes below 10^7.
Time: $\mathcal{O}(N \log \log N)$

bfa652, 11 lines

```
const int MAXP = 1e7;
vector<bool> isPrime(MAXP + 1, true);
vi primes;
void sieve() {
  isPrime[0] = isPrime[1] = false;
  rep(i,2,MAXP+1) if (isPrime[i]) {
    primes.push_back(i);
    for (ll j = (ll)i * i; j <= MAXP; j += i)
      isPrime[j] = false;
  }
}
```

7.2 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2),\ \ b = k \cdot (2mn),\ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

Combinatorial (8)

8.1 Permutations

8.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.) Maps a permutation of 0..n-1 to a unique integer in [0, n!), e.g. to index visited states in a BFS over permutations. Fits in an int for n <= 12. Integer -> permutation can use a lookup table.

Time: $\mathcal{O}(n)$

044568, 6 lines

```
int permToInt(vi& v) {
  int use = 0, i = 0, r = 0;
  for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
    use |= 1 << x;                    // (note: minus, not ~!)
  return r;
}
```

Graph (9)

9.1 Fundamentals

GraphBasics.cpp

Description: Graph as adjacency lists g. bfs(s) returns the unweighted shortest distances from s, -1 if unreachable. dfs(u) marks everything reachable from u in vis. Its recursion can get n deep, and 8 MB of stack fits only about $4 \cdot 10^4$ levels (Codeforces gives 256 MB); bfs also works for reachability. Usage counts connected components.

Usage: `int c = 0; rep(u,0,n) if (!vis[u]) c++, dfs(u);`

Time: $\mathcal{O}(n + m)$

46bed4, 27 lines

```
const int MAXN = 2e5 + 5;
int n, m; // nodes 0..n-1, m edges
vector<int> g[MAXN];
void readGraph() {
  cin >> n >> m;
  rep(i,0,m) {
    int a, b; cin >> a >> b; a--, b--; // if 1-indexed
    g[a].push_back(b);
    g[b].push_back(a); // remove if directed
  }
}
vi bfs(int s) {
  vi dist(n, -1);
  queue<int> q;
  q.push(s), dist[s] = 0;
  while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : g[u]) if (dist[v] < 0)
      dist[v] = dist[u] + 1, q.push(v);
  }
  return dist;
}
bool vis[MAXN];
void dfs(int u) {
  vis[u] = true;
  for (int v : g[u]) if (!vis[v]) dfs(v);
}
```

Bipartite.cpp

Description: Checks whether the undirected graph g can be 2-coloured so that every edge joins two different colours (e.g. split people into two teams with no rivals in the same team). g and n are the globals of GraphBasics.cpp, e.g. filled by readGraph (do not declare your own n). This holds iff there is no cycle of odd length. BFS from each uncoloured node gives each neighbour the other colour; afterwards col[u] is 0 or 1.

Usage: `if (bipartite()) ... col[u] ...`

Time: $\mathcal{O}(n + m)$

"GraphBasics.cpp" 1f0b77, 16 lines

```
vi col;
bool bipartite() {
  col.assign(n, -1);
  rep(s,0,n) if (col[s] < 0) {
    queue<int> q;
    q.push(s), col[s] = 0;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int v : g[u]) {
        if (col[v] < 0) col[v] = !col[u], q.push(v);
        else if (col[v] == col[u]) return false;
      }
    }
  }
  return true;
}
```

TopoSort.h

Description: Topological sorting. Orders the nodes of a directed graph so that every edge goes from left to right, e.g. so that each task comes after the tasks it depends on. gr[u] lists the nodes that u has an edge to. Works by repeatedly taking a node with no incoming edges left. If there is a cycle, the result has fewer than n nodes, so this also detects cycles.

Usage: `vi order = topoSort(vector<vi>(g, g + n));`

Time: $\mathcal{O}(|V| + |E|)$

d678d8, 8 lines

```
vi topoSort(const vector<vi>& gr) {
  vi indeg(sz(gr)), q;
  for (auto& li : gr) for (int x : li) indeg[x]++;
  rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
  rep(j,0,sz(q)) for (int x : gr[q[j]])
    if (--indeg[x] == 0) q.push_back(x);
  return q;
}
```

Cycles. A directed graph has a cycle iff `topoSort` returns fewer than n nodes. An undirected graph has a cycle iff, adding its edges one by one to a `UF`, some `join` returns false.

9.2 Shortest paths

Unweighted: `bfs`. Weights ≥ 0: Dijkstra. Negative weights: Bellman–Ford. All pairs, $n \leq 500$: Floyd–Warshall.

Dijkstra.cpp

Description: Shortest distances from s when no edge weight is negative. adj[u] holds a pair (v, w) for each edge from u to v with weight w (add both directions if undirected). Unreachable nodes get LLONG_MAX. The priority queue always gives the closest node that is not done yet. If you only need s to t, pass t to stop early: then only dist[t] and path(t) are final. par[v] is the node before v on a shortest path, and path(t) follows it back to s.

Usage: `adj[a].push_back({b, w});`
`dijkstra(s, t); ll d = dist[t]; vi p = path(t);`

Time: $\mathcal{O}((n + m) \log m)$

"GraphBasics.cpp" 685eda, 26 lines

```
typedef pair<ll, int> pli;
vector<pair<int, ll>> adj[MAXN];
vector<ll> dist;
vi par;
void dijkstra(int s, int t = -1) {
  dist.assign(n, LLONG_MAX), par.assign(n, -1);
  priority_queue<pli, vector<pli>, greater<pli>> pq;
  dist[s] = 0;
  pq.push({0, s});
  while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue; // u was already done
    if (u == t) break;         // dist[t] is final
    for (auto [v, w] : adj[u]) if (d + w < dist[v]) {
      dist[v] = d + w, par[v] = u;
      pq.push({dist[v], v});
    }
  }
}
vi path(int t) { // s, ..., t, or empty if unreachable
  if (dist[t] == LLONG_MAX) return {};
  vi p;
  for (int u = t; u != -1; u = par[u]) p.push_back(u);
  reverse(all(p));
  return p;
}
```

BellmanFord.cpp

Description: Shortest distances from s when some edge weights are negative (otherwise Dijkstra is faster). edges holds every edge as (a, b, w), from a to b with weight w, and the global n of GraphBasics.cpp must be the number of nodes. For an undirected edge add both directions, but then a negative edge is itself a negative cycle. For real weights (e.g. $-\log$ of exchange rates), use double for w and dist, and only count improvements over 10^{-9}. Each round relaxes every edge. A shortest path has at most n - 1 edges, so without a negative cycle the distances stop changing within n - 1 rounds. Returns false if the n-th round still changes something: then a negative cycle is reachable from s, and some distances are really $-\infty$. Unreachable nodes get LLONG_MAX.

Usage: `n = 5; edges.push_back({a, b, w}); bool ok = bellmanFord(s);`

Time: $\mathcal{O}(nm)$

"GraphBasics.cpp" cc3288, 15 lines

```
struct Edge { int a, b; ll w; };
vector<Edge> edges;
vector<ll> dist;
bool bellmanFord(int s) {
  dist.assign(n, LLONG_MAX);
  dist[s] = 0;
  rep(round,0,n) {
    bool changed = false;
    for (auto [a, b, w] : edges)
      if (dist[a] != LLONG_MAX && dist[a] + w < dist[b])
        dist[b] = dist[a] + w, changed = true;
    if (!changed) return true;
  }
  return false;
```

```
}
```

FloydWarshall.cpp

Description: Shortest distances between all pairs of nodes, for n up to about 500. Before calling, set every d[i][j] = INF and d[i][i] = 0, then d[a][b] = min(d[a][b], w) for each edge (so parallel edges and negative self-loops count). Afterwards d[i][j] is the shortest distance from i to j, or INF if unreachable. The loop over k must be the outermost. Negative weights are fine: there is a negative cycle iff some d[k][k] < 0, and d[i][j] = -INF means the distance is really $-\infty$ (a path from i to j can go round a negative cycle forever).
Time: $\mathcal{O}\left(n^3\right)$

276adf, 10 lines

```
const int MAXV = 505;
const ll INF = 1e18;
ll d[MAXV][MAXV];
void floydWarshall(int n) {
  rep(k,0,n) rep(i,0,n) rep(j,0,n)
    if (d[i][k] < INF && d[k][j] < INF)
      d[i][j] = max(-INF, min(d[i][j], d[i][k] + d[k][j]));
  rep(k,0,n) if (d[k][k] < 0) rep(i,0,n) rep(j,0,n)
    if (d[i][k] < INF && d[k][j] < INF) d[i][j] = -INF;
}
```

9.3 Spanning trees

Kruskal.cpp

Description: Minimum spanning tree: the cheapest set of n - 1 edges that connects all n nodes. Goes through the edges from cheapest to most expensive and keeps each edge that joins two different groups. Returns the total weight; the kept edges are in mst, and the graph is connected iff sz(mst) == n - 1. For a maximum spanning tree, sort in decreasing order.
Usage: `edges.push_back({w, a, b}); ll cost = kruskal(n, edges);`
Time: $\mathcal{O}\left(m \log m\right)$

"../data-structures/UnionFind.h" 4e3562, 11 lines

```
typedef tuple<ll, int, int> WEdge; // (w, a, b)
vector<WEdge> mst;
ll kruskal(int n, vector<WEdge> edges) {
  sort(all(edges));
  UF uf(n);
  mst.clear();
  ll total = 0;
  for (auto [w, a, b] : edges)
    if (uf.join(a, b)) total += w, mst.push_back({w, a, b});
  return total;
}
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1. Otherwise returns (total weight, par), where par[i] is the parent of node i and par[r] = -1.
Time: $\mathcal{O}\left(E \log V\right)$

"../data-structures/UnionFindRollback.h" 39e620, 60 lines

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
}
```

9.4 Trees

LCA.cpp

Description: Lowest common ancestor (LCA) in a tree, by binary lifting. Call build(root) once for the tree in g with n nodes: the globals of GraphBasics.cpp, e.g. filled by readGraph (do not declare your own n). Then up[j][u] is the 2^j-th ancestor of u (the root is its own parent). lca(a, b) lifts the deeper node to the same depth, then lifts both while their ancestors differ. kth(u, k) is the k-th ancestor of u, for $k \leq$ depth[u]. treeDist(a, b) counts the edges between a and b; with weighted edges, store each node's distance d from the root instead and use $d_a + d_b - 2d_{\text{lca}}$.
Usage: `build(0); int c = lca(a, b);`
Time: $\mathcal{O}\left(n \log n\right)$ for build, $\mathcal{O}\left(\log n\right)$ per query

"GraphBasics.cpp" 7cba37, 29 lines

```
const int LOG = 18; // 2^LOG > n
int up[LOG][MAXN], depth[MAXN];
void build(int root) { // BFS, so deep trees are fine
  vi order = {root};
  up[0][root] = root, depth[root] = 0;
  rep(i,0,sz(order)) {
    int u = order[i];
    for (int v : g[u]) if (v != up[0][u]) {
      up[0][v] = u, depth[v] = depth[u] + 1;
      order.push_back(v);
    }
  }
  rep(j,1,LOG) rep(u,0,n) up[j][u] = up[j-1][up[j-1][u]];
}
int kth(int u, int k) {
  rep(j,0,LOG) if (k >> j & 1) u = up[j][u];
  return u;
}
int lca(int a, int b) {
  if (depth[a] < depth[b]) swap(a, b);
  a = kth(a, depth[a] - depth[b]);
  if (a == b) return a;
  for (int j = LOG - 1; j >= 0; j--)
    if (up[j][a] != up[j][b]) a = up[j][a], b = up[j][b];
  return up[0][a];
}
int treeDist(int a, int b) {
  return depth[a] + depth[b] - 2 * depth[lca(a, b)];
}
```

9.5 Flow

Dinic.cpp

Description: Maximum flow from s to t, which must differ (Dinic's algorithm). addEdge(a, b, c) adds a directed edge with capacity c; for an undirected edge, use addEdge(a, b, c, c). Each phase finds the distance of every node from s with BFS (only using edges with capacity left), then pushes flow along shortest paths with DFS. After maxFlow, inCut(u) says whether u is on s's side of a minimum cut, whose capacity equals the max flow. Bipartite matching: an edge from s to each left node, left to right, and right to t, all with capacity 1.
Usage: `Dinic fl(n); fl.addEdge(a, b, c); ll f = fl.maxFlow(s, t);`
Time: $\mathcal{O}\left(V^2E\right)$, and $\mathcal{O}\left(E\sqrt{V}\right)$ for bipartite matching

8be8f9, 44 lines

```
struct Dinic {
  struct E { int to; ll cap; };
  vector<E> e; // e[i ^ 1] is the reverse edge of e[i]
  vector<vi> adj;
  vi level, it;
  Dinic(int n) : adj(n), level(n), it(n) {}
  void addEdge(int a, int b, ll c, ll rc = 0) {
    adj[a].push_back(sz(e)), e.push_back({b, c});
    adj[b].push_back(sz(e)), e.push_back({a, rc});
  }
  bool bfs(int s, int t) {
    fill(all(level), -1);
    queue<int> q;
    q.push(s), level[s] = 0;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int i : adj[u])
        if (e[i].cap > 0 && level[e[i].to] < 0)
          level[e[i].to] = level[u] + 1, q.push(e[i].to);
    }
    return level[t] >= 0;
  }
  ll dfs(int u, int t, ll f) { // push at most f from u to t
    if (u == t) return f;
    for (; it[u] < sz(adj[u]); it[u]++) {
      int i = adj[u][it[u]], v = e[i].to;
      if (e[i].cap <= 0 || level[v] != level[u] + 1) continue;
      if (ll got = dfs(v, t, min(f, e[i].cap))) {
        e[i].cap -= got, e[i ^ 1].cap += got;
        return got;
      }
    }
    return 0;
  }
```

```
  ll maxFlow(int s, int t) {
    ll flow = 0;
    while (bfs(s, t)) {
      fill(all(it), 0);
      while (ll f = dfs(s, t, LLONG_MAX)) flow += f;
    }
    return flow;
  }
  bool inCut(int u) { return level[u] >= 0; }
};
```

Geometry (10)

10.1 Geometric primitives

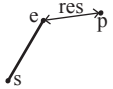
Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

47ec0a, 28 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

SegmentDistance.h



Description:
Returns the shortest distance between point p and the line segment from point s to e.

Usage: `Point<double> a, b(2,2), p(1,1);`
`bool onSegment = segDist(a,b,p) < 1e-10;`

"Point.h" d5f39f, 6 lines

```
typedef Point<double> P;
double segDist(P s, P e, P p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: `bool left = sideOf(p1,p2,q)==1;`

"Point.h" ca4665, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(P s, P e, P p, double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using Point<double>.

"Point.h" c597e8, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

10.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h" 84d6d3, 11 lines

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Returns 0, 1 or 2 tangents as (point on circle 1, point on circle 2), with 0 if one circle contains the other (or they overlap, for internal) and 1 if they touch. For the tangents from a point to a circle, use the point as circle 2 with r2 = 0.

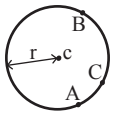
"Point.h" b0153d, 13 lines

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

circumcircle.h

Description:

The circumcircle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



"Point.h" 6cffc5, 9 lines

```
typedef Point<double> P;
double ccRadius(P A, P B, P C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}
P ccCenter(P A, P B, P C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.
Time: expected $\mathcal{O}(n)$

"circumcircle.h" 09dd0a, 17 lines

```
pair<P, double> mec(vector<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      }
    }
  }
  return {o, r};
}
```

10.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`
Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
  }
  return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h" f12300, 6 lines

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

PolygonCenter.h

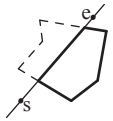
Description: Returns the center of mass for a polygon.
Time: $\mathcal{O}(n)$

"Point.h" 9706dc, 9 lines

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

PolygonCut.h

Description:
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.



Usage: `vector<P> p = ...;`
`p = polygonCut(p, P(0,0), P(1,0));`

"Point.h" d07181, 13 lines

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
  vector<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a - b)));
    if (a < 0)
      res.push_back(cur);
  }
  return res;
}
```

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counterclockwise order. Points on the edge of the hull between two other points are not considered part of the hull.



Time: $\mathcal{O}(n \log n)$

"Point.h" 310954, 13 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points). For the farthest pair of a point set, use hullDiameter(convexHull(pts)).

Time: $\mathcal{O}(n)$

"Point.h" c571b8, 12 lines

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
  rep(i,0,j)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
        break;
    }
  return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 71446b, 14 lines

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)<= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

10.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

"Point.h" ac41a6, 17 lines

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. Use e.g. for Euclidean MST (its edges are among the triangulation's edges, so run Kruskal on those) or nearest neighbours. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

"Point.h" eefdf5, 88 lines

```
typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point

struct Quad {
  Q rot, o; P p = arb; bool mark;
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
  lll p2 = p.dist2(), A = a.dist2()-p2,
      B = b.dist2()-p2, C = c.dist2()-p2;
  return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}
Q makeEdge(P orig, P dest) {
  Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
  H = r->o; r->r()->r() = r;
  rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
  r->p = orig; r->F() = dest;
  return r;
}
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
}

pair<Q,Q> rec(const vector<P>& s) {
  if (sz(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
    if (sz(s) == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = sz(s) / 2;
  tie(ra, A) = rec({all(s) - half});
  tie(B, rb) = rec({sz(s) - half + all(s)});
  while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
         (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    }
  for (;;) {
    DEL(LC, base->r(), o);  DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  }
  return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
  sort(all(pts));  assert(unique(all(pts)) == pts.end());
  if (sz(pts) < 2) return {};
```

```
  Q e = rec(pts).first;
  vector<Q> q = {e};
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
  q.push_back(c->r()); c = c->next(); } while (c != e); }
  ADD; pts.clear();
  while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
  return pts;
}
```

10.5 3D

sphericalDistance.h

Description: Returns the shortest (great-circle) distance between two points on a sphere with radius radius. f1, f2 are longitudes (ϕ, from the x axis) and t1, t2 are angles from the north pole (θ, from the z axis), all in radians. For latitude ℓ, use $\theta = \pi/2 - \ell$. For cartesian points on the unit sphere, set dx, dy, dz to their difference.

611f07, 8 lines

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
  double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
  double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
  double dz = cos(t2) - cos(t1);
  double d = sqrt(dx*dx + dy*dy + dz*dz);
  return radius*2*asin(d/2);
}
```

Strings (11)

Hashing.cpp

Description: String hashing: after O(n) setup, get(l, r) is a number for the substring s[l..r) (r exclusive), so two substrings can be compared in O(1). Equal strings always get equal hashes. It is probabilistic: different strings can also get equal hashes. Here that is very unlikely, as we work mod the prime $p = 2^{61} - 1$ with a random base B: a comparison is wrong with probability at most $n/p \approx 10^{-13}$. It goes wrong with a small modulus like $10^9 + 7$ (a set of $2 \cdot 10^5$ hashes then has about 20 collisions) or with ull overflow (mod 2^{64}), which Thue–Morse strings break for every base. All Hash objects share B, so substrings of different strings can be compared too. For example, longestRepeat finds the longest substring that appears at least twice (possibly overlapping) in $O(n \log n)$. If some length-L substring repeats, so does a length-(L-1) one, so binary search on L, checking each L by putting the hash of every window of length L in a set. The same idea finds the longest common substring of two strings: put one string's windows in the set and look up the other's.

Usage: `Hash h(s); if (h.get(a, a + len) == h.get(b, b + len)) ...`

Time: $\mathcal{O}(n)$ setup, $\mathcal{O}(1)$ per get

797d52, 38 lines

```
typedef unsigned long long ull;
const ull MOD = (1ULL << 61) - 1;
ull mul(ull a, ull b) { return (ull)((__uint128_t)a * b % MOD);
    }
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().
    count());
const ull B = rng() % (MOD - 1000) + 500;

struct Hash {
  vector<ull> h, pw; // h[i] = hash of s[0..i), pw[i] = B^i
  Hash(const string& s) : h(sz(s) + 1), pw(sz(s) + 1, 1) {
    rep(i,0,sz(s)) {
      h[i+1] = (mul(h[i], B) + (unsigned char)s[i]) % MOD;
      pw[i+1] = mul(pw[i], B);
    }
  }
  ull get(int l, int r) { // hash of s[l..r)
    return (h[r] + MOD - mul(h[l], pw[r - l])) % MOD;
  }
};

string longestRepeat(const string& s) {
  Hash hs(s);
  int best = 0, at = 0;
  auto twice = [&](int L) { // a repeat of length L?
    unordered_set<ull> seen;
    rep(i,0,sz(s)-L+1)
      if (!seen.insert(hs.get(i, i + L)).second) {
        at = i;
        return true;
      }
    return false;
  };
  for (int lo = 1, hi = sz(s) - 1; lo <= hi;) {
    int mid = (lo + hi) / 2;
    if (twice(mid)) best = mid, lo = mid + 1;
    else hi = mid - 1;
  }
  return s.substr(at, best);
}
```

KMP.cpp

Description: Knuth–Morris–Pratt. pi[i] is the length of the longest proper prefix of s[0..i] that is also a suffix of it. match(text, pat) returns the start of every occurrence of pat in text, overlapping ones included: in pi of pat + separator + text, a value equal to the length of pat marks the end of a match. The separator is char 0, so neither string may contain it (e.g. numbers stored as chars): pick another one then. The shortest period of s is $p = n -$ pi[n-1], and s is a shorter block repeated iff $p < n$ and p divides n.

Usage: `vi starts = match(text, pat);`

Time: $\mathcal{O}(n + m)$

c8ed04, 16 lines

```
vi prefixFunction(const string& s) {
  vi pi(sz(s));
  rep(i,1,sz(s)) {
    int k = pi[i-1]; // try to extend the previous match
    while (k > 0 && s[i] != s[k]) k = pi[k-1];
    if (s[i] == s[k]) k++;
    pi[i] = k;
  }
  return pi;
}
vi match(const string& text, const string& pat) {
  vi pi = prefixFunction(pat + '\0' + text), res;
  int p = sz(pat);
  rep(i,2*p,sz(pi)) if (pi[i] == p) res.push_back(i - 2*p);
  return res;
}
```

Manacher.h

Description: Use for longest palindromic substring (or all palindromes). p[1][i] = k means s[i-k..i+k] is the longest odd palindrome centred at i, p[0][i] = k means s[i-k..i+k-1] is the longest even palindrome centred between i-1 and i. The longest is the max of 2p[1][i]+1 and 2p[0][i].

Time: $\mathcal{O}(N)$

e7ad79, 13 lines

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```

Various (12)

12.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).

Time: $\mathcal{O}(\log N)$

edce47, 23 lines

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}

void removeInterval(set<pii>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

12.2 Misc. algorithms

LIS.h

Description: Compute indices for the longest increasing subsequence.

Time: $\mathcal{O}(N \log N)$

2932a0, 17 lines

```
template<class I> vi lis(const vector<I>& S) {
  if (S.empty()) return {};
  vi prev(sz(S));
  typedef pair<I, int> p;
  vector<p> res;
  rep(i,0,sz(S)) {
    // change 0 -> i for longest non-decreasing subsequence
    auto it = lower_bound(all(res), p{S[i], 0});
    if (it == res.end()) res.emplace_back(), it = res.end()-1;
    *it = {S[i], i};
    prev[i] = it == res.begin() ? 0 : (it-1)->second;
  }
  int L = sz(res), cur = res.back().second;
  vi ans(L);
  while (L--) ans[L] = cur, cur = prev[cur];
  return ans;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.

Time: $\mathcal{O}(N \max(w_i))$

b20ccc, 16 lines

```
int knapsack(vi w, int t) {
  int a = 0, b = 0, x;
```

```
  while (b < sz(w) && a + w[b] <= t) a += w[b++];
  if (b == sz(w)) return a;
  int m = *max_element(all(w));
  vi u, v(2*m, -1);
  v[a+m-t] = b;
  rep(i,b,sz(w)) {
    u = v;
    rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]);
    for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v[x])
      v[x-w[j]] = max(v[x-w[j]], j);
  }
  for (a = t; v[a+m-t] < 0; a--) ;
  return a;
}
```

12.3 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.

SubsetSums.cpp

Description: Sums over subsets: afterwards D[m] is the sum of the old D[s] over all submasks s of m (masks using only bits of m). The example counts, for each mask m, how many numbers in a fit inside m, e.g. D[5] = 2 as 1 and 4 fit in 101. For sums over supersets, use `if (!(i & 1 << b)) D[i] += D[i | 1 << b]` instead. The same loops with -= undo it, giving back the old D.

Time: $\mathcal{O}\left(K \cdot 2^K\right)$

99d1ee, 7 lines

```
const int K = 3;
vector<int> a = {1, 3, 3, 4};
vector<ll> D(1 << K, 0);
for (int x : a) D[x]++;       // D = [0, 1, 0, 2, 1, 0, 0, 0]
rep(b,0,K) rep(i,0,(1 << K))
  if (i & 1 << b) D[i] += D[i^(1 << b)];
// now D = [0, 1, 0, 3, 1, 2, 1, 4]
```